



MariaDB – Data Manipulation Language

Stephan Karrer

Datenmanipulation (DML)

- Zeilen hinzufügen, Werte verändern, Zeilen löschen
- Data Manipulation Language Statements:
 - INSERT,
 - UPDATE,
 - DELETE,
 - MERGE

INSERT: Neue Zeilen einfügen

```
INSERT INTO departments (department_id, manager_id,  
                        location_id, department_name)  
VALUES ( 70, 100, 1700, 'Public Relations' );
```

```
INSERT INTO departments      --Spaltenreihenfolge!  
VALUES ( 70, 'Public Relations', NULL, NULL);
```

- Sofern die Spaltenliste angegeben wird, werden die Spalten gemäß dieser Reihenfolge befüllt.
- Selbstverständlich müssen die Daten zu den Datentypen und Constraints der Tabelle passen.

INSERT: Spezielles

```
CREATE OR REPLACE TABLE MyEmps (  
  Id INTEGER PRIMARY KEY,  
  FirstName VARCHAR(50),  
  LastName VARCHAR(50) NOT NULL,  
  FullName VARCHAR(101) AS (CONCAT(FirstName, ' ', LastName)),  
  Salary DECIMAL(8,2) DEFAULT 0.0  
);  
  
INSERT INTO myemps (id, lastname) VALUES (10, 'Karrer');  
  
INSERT INTO myemps (id, lastname, fullname)  
  VALUES (12, 'Meier', 'Max Muster') -- Fehler
```

- Schlüsselspalten (sofern nicht automatisch inkrementiert) und Not Null - Spalten müssen befüllt werden!
- Generierte (Computed) Spalten können nicht gesetzt werden!
Das gilt nur teilweise für AutoIncrement-Spalten für die Schlüsselvergabe

Einfügen mehrerer Zeilen

```
CREATE TABLE my_departments ( id          DECIMAL(4,0),
                                name        VARCHAR(30) );

INSERT INTO my_departments ( id, name)
VALUES  ( 1 , 'Abteilung1'),
        ( 2 , 'Abteilung2'),
        ( 3 , 'Abteilung3');

INSERT INTO my_departments (id, name)
SELECT department_id, department_name
   FROM departments
  WHERE manager_id = 100;
```

- Selbstverständlich müssen auch hier die einzufüllenden Daten zu den Datentypen und Constraints der Tabelle passen.

UPDATE: Vorhandene Zeilen ändern

```
UPDATE  employees
        SET department_id = 70,
            manager_id = 100
        WHERE employee_id = 137 ;

UPDATE  my_employees
        SET department_id = 70 ;
```

- Selbstverständlich müssen auch hier die einzufüllenden Daten zu den Datentypen und Constraints der Tabelle passen.
- Vorsicht: Ohne Filter werden alle Zeilen aktualisiert !
- Das Schlüsselwort DEFAULT kann auch hier für den neuen WERT benutzt werden.

UPDATE: Spalten mit Unterabfragen aktualisieren

```
UPDATE employees
  SET  job_id = (SELECT job_id FROM employees
                  WHERE employee_id = 207),
      department_id = (SELECT department_id
                       FROM departments
                       WHERE department_name = 'IT')
 WHERE employee_id = 67;
```

DELETE: Zeilen löschen

```
DELETE FROM employees WHERE employee_id = 67;
```

```
DELETE FROM my_employee;
```

```
DELETE FROM employees  
      WHERE department_id = (SELECT department_id  
                             FROM departments  
                             WHERE department_name =  
                                'Public Relations' );
```

- Vorsicht: Ohne Filter werden alle Zeilen gelöscht !

Transaktionskonzept bei DBMS

- Eine Transaktion umfasst eine oder mehrere Anweisungen für die gilt:

Atomic	Entweder alle Anweisungen sind erfolgreich oder keine
Consistent	Eine erfolgreiche Transaktion führt den Datenbestand in einen konsistenten Zustand (semantischer Begriff!).
Isolated	Die Zwischenzustände des Datenbestands während einer Transaktion sind für parallel laufende Zugriffe nicht sichtbar.
Durable	Die Ergebnisse einer erfolgreichen Transaktion werden in der Datenbank persistiert.
- ANSI fordert: Alle schreibenden Zugriffe müssen innerhalb einer Transaktion erfolgen.
 - dies betrifft auf jeden Fall alle DML-Anweisungen.
- Bei allen Systemen gilt:

Eine einzelne SQL-Anweisung ist auf jeden Fall transaktional.

Umsetzung des Transaktionskonzepts

- Da die Anweisungsfolge innerhalb einer Transaktion anforderungsspezifisch ist:
COMMIT für die explizite erfolgreiche Beendigung
ROLLBACK für den Abbruch (und damit rückgängig machen aller Änderungen)
- Einige Systeme, z.B. Oracle, DB2 benutzen implizite Transaktionssteuerung:
Eine neue Transaktion startet automatisch, wenn die vorherige Transaktion explizit oder implizit durch das System beendet wird und umfasst jetzt alle folgenden Anweisungen.
- Andere Systeme (PostgreSQL, SQL Server, MariaDB) nutzen „AutoCommit“-Modus:
Standardmäßig ist nur eine einzelne Anweisung eine Transaktion. Sollen mehrere Anweisungen in einer Transaktionsklammer ausgeführt werden, so muss diese explizit gestartet werden:
 - START TRANSACTION (ANSI) , herstellerspezifische Anweisungen sind auch üblich!

Transaktionsteuerung am Bsp. MariaDB

```
-- Ende der letzten Transaktion
START TRANSACTION;
INSERT INTO departments
    VALUES (280, 'Recreation', 110, 1700);
UPDATE employees SET salary = 10 WHERE employee_id = 100;
SELECT * FROM employees WHERE employee_id =100;
ROLLBACK;
SELECT * FROM employees WHERE employee_id =100;

-- jetzt sind wir wieder im AutoCommit-Modus
-- nächste Transaktion beginnt
START TRANSACTION;
UPDATE employees SET salary = 48000 WHERE employee_id = 100;
SAVEPOINT punkt1;
UPDATE employees SET salary = 200 WHERE employee_id = 100;
SELECT * FROM employees WHERE employee_id =100;
ROLLBACK TO SAVEPOINT punkt1;
SELECT * FROM employees WHERE employee_id =100;
COMMIT;
SELECT * FROM employees WHERE employee_id =100;
-- jetzt sind wir wieder im AutoCommit-Modus
UPDATE employees SET salary = 24000 WHERE employee_id = 100;
```